

FIAP

CIÊNCIAS DA COMPUTAÇÃO

MARIA LUIZA COSTA ARAUJO

Missão Aurora Siger

Sistema de Verificação de Pré-Decolagem

SETEMBRO, 2026.

1.1 Organização e descrição da telemetria

Faixas seguras adotadas:

- Temperatura interna: 15°C a 30°C
- Temperatura externa: -50°C a 50°C
- Integridade estrutural: = 1
- Nível de energia: $\geq 85\%$
- Pressão de cada tanque: 28 a 32 bar
- Módulos críticos (nav, comunicação, suporte de vida, propulsão): = 1 (todos)

1.2 Algoritmo de verificação

O algoritmo decide entre "**PRONTO PARA DECOLAR**" e "**DECOLAGEM ABORTADA**", comparando cada leitura com faixas seguras predefinidas.

Pseudocódigo:

PARA cada leitura de telemetria:

 falhas <- lista vazia

 SE temp_interna fora de [15, 30] ENTÃO adicionar falha "temperatura"

 SE integridade != 1 ENTÃO adicionar falha "integridade"

 SE energia < 85% ENTÃO adicionar falha "energia"

 PARA cada tanque: SE pressão fora de [28, 32] ENTÃO adicionar falha "pressão"

 PARA cada módulo crítico: SE status != 1 ENTÃO adicionar falha "módulo crítico"

 SE falhas está vazia ENTÃO veredito <- "PRONTO PARA DECOLAR"

 SENÃO veredito <- "DECOLAGEM ABORTADA"

IMPRIMIR veredito e falhas

1.3 Script em Python

```
FAIXAS_SEGURAS = {
    "temp_interna": (15.0, 30.0),
    "temp_externa": (-50.0, 50.0),
    "energia_min": 85,
    "pressao_tanque": (28.0, 32.0),
    "modulos_criticos": ["nav", "comunicacao", "suporte_vida", "propulsao"],
}

def verificar_leitura(leitura):
    falhas = []

    t_min, t_max = FAIXAS_SEGURAS["temp_interna"]
    if not (t_min <= leitura["temp_interna"] <= t_max):
        falhas.append(f"Temperatura interna fora da faixa ({leitura['temp_interna']}°C)")

    te_min, te_max = FAIXAS_SEGURAS["temp_externa"]
    if not (te_min <= leitura["temp_externa"] <= te_max):
        falhas.append(f"Temperatura externa fora da faixa ({leitura['temp_externa']}°C)")

    if leitura["integridade"] != 1:
        falhas.append("Integridade estrutural comprometida")

    if leitura["energia"] < FAIXAS_SEGURAS["energia_min"]:
        falhas.append(f"Energia abaixo do mínimo ({leitura['energia']}%)")

    p_min, p_max = FAIXAS_SEGURAS["pressao_tanque"]
    for tanque in ("pressao_t1", "pressao_t2"):
        if not (p_min <= leitura[tanque] <= p_max):
            falhas.append(f"Pressão do {tanque} fora da faixa ({leitura[tanque]} bar)")

    for modulo in FAIXAS_SEGURAS["modulos_criticos"]:
        if leitura[modulo] != 1:
            falhas.append(f"Módulo crítico '{modulo}' com falha")

    veredito = "DECOLAGEM ABORTADA" if falhas else "PRONTO PARA DECOLAR"
```

```
return {"T_min": leitura["T_min"], "veredito": veredito, "falhas": falhas}
```

```
def formatar_rotulo_tempo(t_min):
```

```
    if t_min == 0:
```

```
        return "T-0min"
```

```
    sinal = "+" if t_min > 0 else "-"
```

```
    return f"T{sinal}{abs(t_min)}min"
```

```
def main():
```

```
    # Cenário de sucesso: todas as leituras dentro das faixas seguras
```

```
    telemetria = [
```

```
        {
```

```
            "T_min": -10, "temp_interna": 22.0, "temp_externa": -20.0,
```

```
            "integridade": 1, "energia": 90,
```

```
            "pressao_t1": 30.0, "pressao_t2": 31.0,
```

```
            "nav": 1, "comunicacao": 1, "suporte_vida": 1, "propulsao": 1
```

```
        },
```

```
        {
```

```
            "T_min": -5, "temp_interna": 27.0, "temp_externa": -15.0,
```

```
            "integridade": 1, "energia": 88,
```

```
            "pressao_t1": 29.0, "pressao_t2": 32.0,
```

```
            "nav": 1, "comunicacao": 1, "suporte_vida": 1, "propulsao": 1
```

```
        },
```

```
        {
```

```
            "T_min": 0, "temp_interna": 25.0, "temp_externa": -10.0,
```

```
            "integridade": 1, "energia": 95,
```

```
            "pressao_t1": 30.0, "pressao_t2": 30.0,
```

```
            "nav": 1, "comunicacao": 1, "suporte_vida": 1, "propulsao": 1
```

```
        }
```

```
    ]
```

```
    print("=== EXECUÇÃO DAS VERIFICAÇÕES ===")
```

```
    for linha in telemetria:
```

```
        resultado = verificar_leitura(linha)
```

```
        print(f'{formatar_rotulo_tempo(resultado["T_min"])} -> {resultado["veredito"]}')"
```

```
    for falha in resultado["falhas"]:
```

```
        print("  -", falha)
```

```
print("\n=== RESULTADO FINAL (T-0) ===")
final = verificar_leitura(telemetry[-1])
print(final["veredito"])
```

```
if __name__ == "__main__":
    main()
```

=== EXECUÇÃO DAS VERIFICAÇÕES ===

T-10min -> PRONTO PARA DECOLAR

T-5min -> PRONTO PARA DECOLAR

T-0min -> PRONTO PARA DECOLAR

=== RESULTADO FINAL (T-0) ===

PRONTO PARA DECOLAR

```
FAIXAS_SEGURAS = {
    "temp_interna": (15.0, 30.0),
    "temp_externa": (-50.0, 50.0),
    "energia_min": 85,
    "pressao_tanque": (28.0, 32.0),
    "modulos_criticos": ["nav", "comunicacao", "suporte_vida", "propulsao"],
}
```

```
def verificar_leitura(leitura):
```

```
    falhas = []
```

```
    t_min, t_max = FAIXAS_SEGURAS["temp_interna"]
```

```
    if not (t_min <= leitura["temp_interna"] <= t_max):
```

```
        falhas.append(f"Temperatura interna fora da faixa ({leitura['temp_interna']}°C)")
```

```
    te_min, te_max = FAIXAS_SEGURAS["temp_externa"]
```

```
    if not (te_min <= leitura["temp_externa"] <= te_max):
```

```
        falhas.append(f"Temperatura externa fora da faixa ({leitura['temp_externa']}°C)")
```

```
    if leitura["integridade"] != 1:
```

```
        falhas.append("Integridade estrutural comprometida")
```

```
    if leitura["energia"] < FAIXAS_SEGURAS["energia_min"]:
```

```

falhas.append(f"Energia abaixo do mínimo ({leitura['energia']}%)")

p_min, p_max = FAIXAS_SEGURAS["pressao_tanque"]
for tanque in ("pressao_t1", "pressao_t2"):
    if not (p_min <= leitura[tanque] <= p_max):
        falhas.append(f"Pressão do {tanque} fora da faixa ({leitura[tanque]} bar)")

for modulo in FAIXAS_SEGURAS["modulos_criticos"]:
    if leitura[modulo] != 1:
        falhas.append(f"Módulo crítico '{modulo}' com falha")

veredito = "DECOLAGEM ABORTADA" if falhas else "PRONTO PARA DECOLAR"
return {"T_min": leitura["T_min"], "veredito": veredito, "falhas": falhas}

def formatar_rotulo_tempo(t_min):
    if t_min == 0:
        return "T-0min"
    sinal = "+" if t_min > 0 else "-"
    return f"T{sinal}{abs(t_min)}min"

def main():
    # Cenário de falha: temperatura interna sobe em T-5min,
    # e integridade + pressão do tanque 1 falham em T-0min
    telemetria = [
        {
            "T_min": -10, "temp_interna": 22.0, "temp_externa": -20.0,
            "integridade": 1, "energia": 90,
            "pressao_t1": 30.0, "pressao_t2": 31.0,
            "nav": 1, "comunicacao": 1, "suporte_vida": 1, "propulsao": 1
        },
        {
            "T_min": -5, "temp_interna": 31.0, "temp_externa": -15.0,
            "integridade": 1, "energia": 88,
            "pressao_t1": 29.0, "pressao_t2": 32.0,
            "nav": 1, "comunicacao": 1, "suporte_vida": 1, "propulsao": 1
        },
        {
            "T_min": 0, "temp_interna": 25.0, "temp_externa": -10.0,

```

```

        "integridade": 0, "energia": 95,
        "pressao_t1": 27.0, "pressao_t2": 30.0,
        "nav": 1, "comunicacao": 1, "suporte_vida": 1, "propulsao": 1
    }
]

```

```

print("=== EXECUÇÃO DAS VERIFICAÇÕES ===")
for linha in telemetria:
    resultado = verificar_leitura(linha)
    print(f"{formatar_rotulo_tempo(resultado['T_min'])} -> {resultado['veredito']}")
    for falha in resultado["falhas"]:
        print("  -", falha)

```

```

print("\n=== RESULTADO FINAL (T-0) ===")
final = verificar_leitura(telemetria[-1])
print(final["veredito"])

```

```

if __name__ == "__main__":
    main()

```

=== EXECUÇÃO DAS VERIFICAÇÕES ===

T-10min -> PRONTO PARA DECOLAR

T-5min -> DECOLAGEM ABORTADA

- Temperatura interna fora da faixa (31.0°C)

T-0min -> DECOLAGEM ABORTADA

- Integridade estrutural comprometida

- Pressão do pressao_t1 fora da faixa (27.0 bar)

=== RESULTADO FINAL (T-0) ===

DECOLAGEM ABORTADA

1.4 Análise energética

Cálculo da autonomia inicial a partir de: capacidade total, carga atual, consumo estimado na decolagem e perdas energéticas.

$$\text{energia_disponivel} = \text{capacidade_total_kwh} \times (\text{carga_atual_} \% / 100)$$
$$\text{perdas_kwh} = \text{energia_disponivel} \times (\text{perdas_} \% / 100)$$
$$\text{energia_liquida} = \text{energia_disponivel} - \text{consumo_decolagem_kwh} - \text{perdas_kwh}$$
$$\text{autonomia_horas} = \text{energia_liquida} / \text{consumo_cruzeiro_kwh_por_hora}$$

```
def calcular_autonomia(capacidade_total_kwh, carga_atual_pct, consumo_decolagem_kwh,
                        perdas_pct, consumo_cruzeiro_kwh_h):
    energia_disponivel = capacidade_total_kwh * (carga_atual_pct / 100)
    perdas_kwh = energia_disponivel * (perdas_pct / 100)
    energia_liquida = energia_disponivel - consumo_decolagem_kwh - perdas_kwh
    autonomia_horas = energia_liquida / consumo_cruzeiro_kwh_h
    return {
        "energia_disponivel": energia_disponivel,
        "perdas_kwh": perdas_kwh,
        "energia_liquida": energia_liquida,
        "autonomia_horas": autonomia_horas,
    }
```

Dados de exemplo da missão Aurora Siger

```
capacidade_total_kwh = 500

carga_atual_pct = 92

consumo_decolagem_kwh = 40

perdas_pct = 4

consumo_cruzeiro_kwh_h = 6.5


resultado = calcular_autonomia(

    capacidade_total_kwh, carga_atual_pct, consumo_decolagem_kwh,

    perdas_pct, consumo_cruzeiro_kwh_h

)


print(f'Energia disponível : {resultado['energia_disponivel']:.2f} kWh")

print(f'Perdas energéticas : {resultado['perdas_kwh']:.2f} kWh")

print(f'Energia líquida    : {resultado['energia_liquida']:.2f} kWh")

print(f'Autonomia estimada : {resultado['autonomia_horas']:.2f} horas")
```

```
Energia disponível : 460.00 kWh
Perdas energéticas : 18.40 kWh
Energia líquida    : 401.60 kWh
Autonomia estimada : 61.78 horas
```

1.5 Análise assistida por IA

Classificação dos dados

As leituras cobrem 9 parâmetros por instante (temperatura interna, temperatura externa, integridade, energia, pressão dos tanques 1 e 2, e 4 módulos críticos), medidos em três marcos da contagem regressiva: T-10min, T-5min e T-0min.

- **Cenário 1 (sucesso):** todas as 27 medições (9 parâmetros × 3 instantes) permanecem dentro das faixas seguras. Nenhuma falha registrada em nenhum dos três momentos.
- **Cenário 2 (falha):** a leitura em T-10min está totalmente normal. A partir de T-5min, um parâmetro sai da faixa (temperatura interna). Em T-0min, dois parâmetros falham simultaneamente (integridade estrutural e pressão do tanque 1), enquanto os demais 7 parâmetros permanecem normais.

Identificação de possíveis anomalias

- No Cenário 2, a temperatura interna sobe de 22,0°C (T-10min) para 31,0°C (T-5min) — uma variação de +9°C em 5 minutos, que ultrapassa o limite de 30°C por uma margem pequena (apenas 1°C acima do teto). Isso é mais consistente com um evento térmico transitório (falha pontual de refrigeração) do que com um problema estrutural grave, já que a temperatura não é reavaliada de volta antes de T-0min neste cenário.
- Em T-0min, a queda de integridade estrutural (de 1 para 0) ocorre no mesmo instante da queda de pressão no tanque 1 (de 29,0 para 27,0 bar). A concorrência dessas duas falhas é o ponto mais crítico da simulação: uma perda de pressão pode ser causa ou consequência de dano estrutural, o que sugere investigar se há uma relação de causalidade entre elas, e não tratá-las como duas falhas independentes.
- Chama atenção que a temperatura externa permanece estável e dentro da faixa segura (-20°C a -10°C) em ambos os cenários, o que descarta condições ambientais externas como causa das falhas do Cenário 2 — o problema é interno à espaçonave.

Sugestões de risco

- Tratar qualquer falha de integridade estrutural como bloqueante absoluto para decolagem, independentemente do estado dos demais parâmetros — o algoritmo já faz isso corretamente, já que uma falha isolada já é suficiente para abortar.
- Investigar a causa raiz da queda de pressão no tanque 1 antes de qualquer nova tentativa, verificando possível vazamento ou falha de vedação, dado que essa falha coincide com a de integridade.
- Adicionar ao sistema um histórico de variação (não só o valor pontual de cada leitura) para detectar tendências perigosas — como o salto de +9°C em temperatura interna entre T-10min e T-5min — antes que a variável efetivamente ultrapasse o limite seguro, permitindo um alerta preventivo em vez de reativo.

1.6 Reflexão crítica

Ética e responsabilidade

Um algoritmo que decide entre decolar ou abortar carga responsabilidade direta sobre vidas humanas e recursos de altíssimo custo, o que aproxima essa decisão técnica dos princípios de Responsabilidade Social Empresarial descritos na ISO 26000 (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2010). Dois desses princípios são especialmente aplicáveis aqui: *accountability*, o dever de prestar contas sobre as próprias ações a todos os interessados, e a transparência sobre atividades que geram impacto. Um sistema de verificação de decolagem só cumpre esses princípios se permanecer auditável — como mostrado pela simulação em T-0min, em que a queda de pressão no tanque 1 e a falha de integridade estrutural só puderam ser investigadas e explicadas porque os dados brutos e a lógica de verificação permaneceram visíveis, e não escondidos numa "caixa-preta" de decisão automática. Definir as faixas seguras (temperatura, pressão, integridade) não é, portanto, uma escolha puramente técnica: é uma decisão que precisa ser rastreável e justificável perante quem depende dela.

Impacto social da exploração espacial

O conceito de *Triple Bottom Line* — o tripé formado por *People*, *Planet* e *Profit* (D'HONT, 2019) — ajuda a estruturar essa discussão. Historicamente, missões espaciais produzem avanços tecnológicos que retornam à sociedade — telecomunicações, materiais mais leves e resistentes, sistemas de monitoramento climático — e mesmo protocolos de verificação de segurança, como o algoritmo desta atividade, que podem migrar para outras indústrias de alto risco. Assim como a sustentabilidade "não fica em pé" se um dos três pilares do tripé for ignorado (D'HONT, 2019), uma missão espacial só se sustenta socialmente se equilibrar ambição exploratória com retorno concreto e transparência sobre custos e benefícios para a sociedade.

Sustentabilidade tecnológica

A Segunda Lei da Termodinâmica estabelece que nenhum sistema alcança eficiência total, pois parte da energia sempre se converte em formas menos úteis (MORAN; SHAPIRO, 2018 apud Cap. 7). Esse limite físico se aplica tanto a uma nave quanto a um data center: por isso métricas como o PUE (*Power Usage Effectiveness*) nunca atingem o valor ideal de 1,0 na prática, operando entre 1,2 e 1,4 mesmo nas instalações mais eficientes (THE GREEN GRID, 2023 apud Cap. 7). A análise energética desta atividade ilustra o mesmo princípio em pequena escala: das 500 kWh de capacidade total, cerca de 4% se perderam apenas com as perdas energéticas estimadas, sem contar o consumo da própria decolagem. Em operações contínuas ou missões prolongadas, esse tipo de perda percentual se acumula de forma significativa. Isso reforça uma ideia central do conceito de Green IT: a eficiência energética deve ser tratada como critério de projeto desde a concepção do sistema — o que a Green IT chama de abordagem "estratégica" ou "profunda" —, e não como um ajuste corretivo aplicado depois que o sistema já está em operação.